OS – Banana Problem – Unit 4

Likhith S
PES1UG24CS720
Section – L

Phase 1:

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ make test_objects
gcc -Wall -Wextra -O2 -c object.c -o object.o
object.c: In function 'object_read':
object.c:185:5: warning: ignoring return value of 'fread' declared with attribute 'warn_unused_result' [-Wunused-result]
  185 |     fread(buf, 1, file_size, f);
      |     ^~~~~~~~~~~~~~~~~~~~~~~~~~~
object.c: In function 'object_write':
object.c:131:26: warning: '.tmp' directive writing 4 bytes into a region of size between 1 and 512 [-Wformat-overflow=]
  131 |     sprintf(tmp_path, "%s.tmp", path);
      |                          ^~~~
In file included from /usr/include/stdio.h:894,
                 from pes.h:8,
                 from object.c:11:
/usr/include/x86_64-linux-gnu/bits/stdio2.h:38:10: note: '__builtin___sprintf_chk' output between 5 and 516 bytes into a destination of size 512
   38 |   return __builtin___sprintf_chk (__s, __USE_FORTIFY_LEVEL - 1,
      |          ^~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   39 |                                   __glibc_objsize (__s), __fmt,
      |                                   ~~~~~~~~~~~~~~~~~~~~~~~~~~~~
   40 |                                   __va_arg_pack ());
      |                                   ~~~~~~~~~~~~~~~~~
gcc -o test_objects test_objects.o object.o -lcrypto
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
```

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$
```

Phase 2:

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ make test_tree
make: 'test_tree' is up to date.
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
```

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ xxd .pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf | head -20
00000000: 626c 6f62 2031 3500 5465 7374 2069 6e74  blob 15.Test int
00000010: 6567 7269 5879 0a                        egriXy.
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$
```

Phase 3:

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ echo "test" > hello.txt
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ ./pes add hello.txt
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ ./pes status
Staged changes:
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  object.c
  untracked:  test_objects
  untracked:  Makefile
  untracked:  pes.h
  untracked:  test.txt
  untracked:  myfile.txt
  untracked:  README.md
  untracked:  commit.h
  untracked:  test_tree.c
  untracked:  test_tree
  untracked:  pes.c
  untracked:  index.c
  untracked:  test_objects.c
  untracked:  .index.c.swp
  untracked:  commit.c
  untracked:  test_sequence.sh
  untracked:  tree.c
  untracked:  file1.txt
  untracked:  tree.h
  untracked:  file2.txt
  untracked:  .gitignore
  untracked:  index.h
```

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ cat .pes/index
100755 999f24152159e51756a944d32257bf22080ff8608fff87ca9a4a823764e13dbe 1776352756 5 hello.txt
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$
```

Phase 4:

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ ./pes log
No commits yet.
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ echo "Hello" > hello.txt
./pes add hello.txt
./pes commit -m "Initial commit"

echo "World" >> hello.txt
./pes add hello.txt
./pes commit -m "Add world"

echo "Goodbye" > bye.txt
./pes add bye.txt
./pes commit -m "Add farewell"
Committed: fcd5d06abb4f... Initial commit
Committed: 90237c1e1c87... Add world
Committed: 766a2d310ebb... Add farewell
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ ./pes log
commit 766a2d310ebb7e53c0c00ee484520914f0ab39d70ae07eacbbbdfe7e2fe6f8cf
Author: PES User <pes@localhost>
Date:   1776356906

    Add farewell

commit 90237c1e1c87f9b79354c44c36f1efab1d6458b89c4d730360328a15ff19fe93
Author: PES User <pes@localhost>
Date:   1776356906

    Add worldell

commit fcd5d06abb4fed7678e9f02e6a56e02d3e08387e06f5e914f9399fe7b6a412df
Author: PES User <pes@localhost>
Date:   1776356906

    Initial committial commiQ

likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$
```

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/05/c8d777d0be85748b6788d1b5caf7d69754e6e68060c265ca21cbd6c05e5f43
.pes/objects/55/8c1061e21bccbeb23c527d8dcef5c8646cabfb6091aae3ae7ee3320c9ff7b7
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/76/6a2d310ebb7e53c0c00ee484520914f0ab39d70ae07eacbbbdfe7e2fe6f8cf
.pes/objects/7f/021bbfa48721cd7a4c81e9f88a253ea1c196784ecbd2a8ed36c4f01eb74028
.pes/objects/90/237c1e1c87f9b79354c44c36f1efab1d6458b89c4d730360328a15ff19fe93
.pes/objects/dd/82340b5e403e2dc18bfca050e6699935849710b2b5d28ab8bbffbb72d32627
.pes/objects/e6/7ed66bcb4a708250a89f315d4d8bb92703343c84df834438880e13a68652bc
.pes/objects/fc/d5d06abb4fed7678e9f02e6a56e02d3e08387e06f5e914f9399fe7b6a412df
.pes/refs/heads/main
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$
```

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ cat .pes/refs/heads/main
766a2d310ebb7e53c0c00ee484520914f0ab39d70ae07eacbbbdfe7e2fe6f8cf
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Phase 5:

Q5.1: A branch in Git is just a file in .git/refs/heads/ containing a commit hash. Creating a branch is creating a file. Given this, how would you implement pes checkout <branch> — what files need to change in .pes/, and what must happen to the working directory? What makes this operation complex?

- Files in .pes/ to change: The HEAD file must be updated to point to the new branch reference (e.g., ref: refs/heads/<branch>).
- Working Directory actions: The working directory must be "synchronized" with the snapshot recorded in the target branch's commit. This involves deleting files that are not in the new commit, updating files that have changed, and adding files that exist in the target commit but not the current one.
- Complexity: The operation is complex because it must handle "safety." If the user has uncommitted changes, a naive checkout could overwrite their work. Additionally, updating the working directory requires recursive traversal of tree objects and efficient comparison with the current index.

Q5.2: When switching branches, the working directory must be updated to match the target branch's tree. If the user has uncommitted changes to a tracked file, and that file differs between branches, checkout must refuse. Describe how you would detect this "dirty working directory" conflict using only the index and the object store.

- Detection Logic: You would compare the SHA-256 hash of the file currently in the working directory (calculated on the fly) with the hash stored for that file in the current index. If the hashes differ, the file is "dirty."
- Conflict Check: If a file is dirty, you then compare the index hash with the hash of the same file in the target branch's tree. If the file in the target branch is different from what is in the index, and the working directory version is also different from the index, a conflict exists, and the operation should be aborted to prevent data loss.

Q5.3: "Detached HEAD" means HEAD contains a commit hash directly instead of a branch reference. What happens if you make commits in this state? How could a user recover those commits?

- Committing in Detached HEAD: Commits work normally, and each new commit points to the previous one as its parent. However, since no branch reference (like main) is being updated, there is no "name" attached to these new commits.
- Recovery: If the user switches away to another branch, the hashes of the "detached" commits may be lost from memory. To recover them, the user would need to find the commit hash (often found in the reflog in real Git) and manually create a new branch pointing to that hash using pes branch <name> <hash>.

Phase 6:

Q6.1: Over time, the object store accumulates unreachable objects — blobs, trees, or commits that no branch points to (directly or transitively). Describe an algorithm to find and delete these objects. What data structure would you use to track "reachable" hashes efficiently? For a repository with 100,000 commits and 50 branches, estimate how many objects you'd need to visit.

- Algorithm (Mark and Sweep):
    1. Mark: Start at all known "roots" (every branch in refs/heads/ and the HEAD). Recursively traverse every commit, its tree, and all sub-trees/blobs, adding their hashes to a "reachable" set.
    2. Sweep: Iterate through every file in the .pes/objects/ directory. If an object's hash is not in the "reachable" set, delete it.
- Data Structure: A Hash Set or a Bloom Filter (for initial screening) would be most efficient to track reachable hashes.
- Estimation: You would need to visit all 100,000 commits. Assuming an average of 10-20 unique objects (trees/blobs) per commit across history, you might visit 1 to 2 million objects.

Q6.2: Why is it dangerous to run garbage collection concurrently with a commit operation? Describe a race condition where GC could delete an object that a concurrent commit is about to reference. How does Git's real GC avoid this?

- Danger: GC might identify an object as "unreachable" and delete it just as another process is creating a new commit that intends to use that exact object.
- Race Condition:
    1. Process A (Commit) creates a new blob and writes it to the object store.
    2. Process B (GC) starts. It looks at the current branches, but Process A hasn't updated the branch reference yet.
    3. Process B sees the new blob as "unreferenced" and deletes it.
    4. Process A tries to finalize the commit, but its required blob is now missing.
- Git’s Solution: Real Git uses a "grace period" (the gc.pruneExpire setting). GC will only delete unreachable objects that are older than a certain age (e.g., 2 weeks), ensuring that brand-new objects currently being handled by concurrent processes are safe.

Full integration test:

```
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: c6ed40cd9095... Initial commit

Log after first commit:
commit c6ed40cd9095a8b4b5d16f9789b5441f08010e1beb70e44357cfc0e24fe3a7ca
Author: PES User <pes@localhost>
Date:   1776356996

    Initial commit


--- Second Commit ---
Committed: 53ec109a3222... Update file.txt

--- Third Commit ---
Committed: 931dc9706ad7... Add farewell

--- Full History ---
commit 931dc9706ad7b9928d847842a31a7747e381628f68a870b55dfa830c50897c81
Author: PES User <pes@localhost>
Date:   1776356996

    Add farewell

commit 53ec109a32226cb45b33febea1de504978ad1e2c0be4c24030f9f633d8ae1574
Author: PES User <pes@localhost>
Date:   1776356996

    Update file.txt
```

```
commit c6ed40cd9095a8b4b5d16f9789b5441f08010e1beb70e44357cfc0e24fe3a7ca
Author: PES User <pes@localhost>
Date:   1776356996

    Initial committial commiQ


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
931dc9706ad7b9928d847842a31a7747e381628f68a870b55dfa830c50897c81

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/53/ec109a32226cb45b33febea1de504978ad1e2c0be4c24030f9f633d8ae1574
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/84/c5fed72c2dd9a3130e2e9b98033827f62793fb80d0295ccedf5fc7972d96eb
.pes/objects/93/1dc9706ad7b9928d847842a31a7747e381628f68a870b55dfa830c50897c81
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/c6/ed40cd9095a8b4b5d16f9789b5441f08010e1beb70e44357cfc0e24fe3a7ca
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff
.pes/objects/de/34fb483a5c16ce29ec3195dd505dfda7e35cc5fe16a99da4b2455b332a93cf
.pes/objects/ec/0af37b159ae92a5850ceaa6e497860729d6158aab2c6833b6068a1e3c5e8ef

=== All integration tests completed ===
likhith@ubuntu-vm:~/OS-banana-u4/PES1UG24CS720-pes-vcs$
```